

PERTEMUAN KE-10: EVALUASI METODE CLUSTERING

A. TUJUAN

Setelah menempuh praktikum ini diharapkan mahasiswa mampu

1. Memahami pentingnya evaluasi dalam clustering.
2. Menjelaskan dan menerapkan metode Elbow untuk menentukan jumlah cluster optimal.
3. Menggunakan metode Silhouette untuk menilai kualitas hasil clustering.
4. Mengenal metode evaluasi clustering lainnya seperti Davies-Bouldin Index, Dunn Index, dan lainnya.

B. MATERI

Clustering merupakan teknik **unsupervised learning**, sehingga tidak memiliki label kelas yang benar. Oleh karena itu, evaluasi clustering berbeda dengan supervised learning yang menggunakan metrik seperti akurasi atau F1-score. Evaluasi clustering dapat dilakukan dengan dua pendekatan utama:

- **Evaluasi Internal** → Mengukur seberapa baik struktur cluster tanpa informasi eksternal. Contoh: **Silhouette Score, Davies-Bouldin Index, Dunn Index.**
- **Evaluasi Eksternal** → Membandingkan hasil clustering dengan ground truth (jika tersedia). Contoh: **Rand Index, Mutual Information Score.**
- **Evaluasi Stabilitas** → Mengukur konsistensi hasil clustering jika data sedikit berubah.

Metode Elbow

Definisi : Metode Elbow digunakan untuk menentukan jumlah cluster optimal (k) dalam algoritma seperti K-Means. Metode ini bekerja dengan melihat bagaimana nilai inertia (WCSS - Within-Cluster Sum of Squares) berubah saat jumlah cluster bertambah.

Rumus WCSS (Within-Cluster Sum of Squares):

$$WCSS = \sum_{i=1}^k \sum_{x \in C_i} ||x - \mu_i||^2$$

Dimana:

- x adalah titik data dalam cluster C_i .
- μ_i adalah centroid cluster C_i
- $||x - \mu_i||^2$ adalah kuadrat jarak antara titik data dengan centroid.

Jika jumlah cluster terlalu kecil, WCSS tinggi karena banyak data dalam satu cluster yang tidak mirip. Jika jumlah cluster terlalu besar, WCSS rendah, tetapi bisa menyebabkan overfitting.

Contoh 9.1

Dataset Sederhana (Koordinat 6 Titik Data dalam 2D)

Misalkan kita memiliki dataset 6 titik dalam bidang 2D:

Titik	A	B
A	1	2
B	1	4
C	2	3
D	8	8
E	9	9
F	8	10

Langkah 1: Clustering dengan k = 1 (Semua titik dalam 1 cluster)

Centroid dihitung sebagai rata-rata dari semua titik:

$$\mu = \frac{1 + 1 + 2 + 8 + 9 + 8}{6}, \frac{2 + 4 + 3 + 8 + 9 + 10}{6} = (4.83, 6)$$

$$\begin{aligned}
 WCSS &= (1 - 4.83)^2 + (2 - 6)^2 + (1 - 4.83)^2 + (4 - 6)^2 + (2 - 4.83)^2 + (3 - 6)^2 + (8 - 4.83)^2 \\
 &\quad + (8 - 6)^2 + (9 - 4.83)^2 + (9 - 6)^2 + (8 - 4.83)^2 + (10 - 4.83)^2 \\
 &= 14.7 + 16 + 14.7 + 4 + 7.9 + 9 + 10.1 + 4 + 17.4 + 9 + 10.1 + 16 \\
 &= 133.9
 \end{aligned}$$

Langkah 2: Clustering dengan k = 2

Misalkan kita bagi dataset menjadi 2 cluster:

- Cluster 1: {A, B, C}, centroid = (1.33, 3)
- Cluster 2: {D, E, F}, centroid = (8.33, 9)

Langkah 3: Clustering dengan k = 3

Misalkan dataset dibagi menjadi 3 cluster:

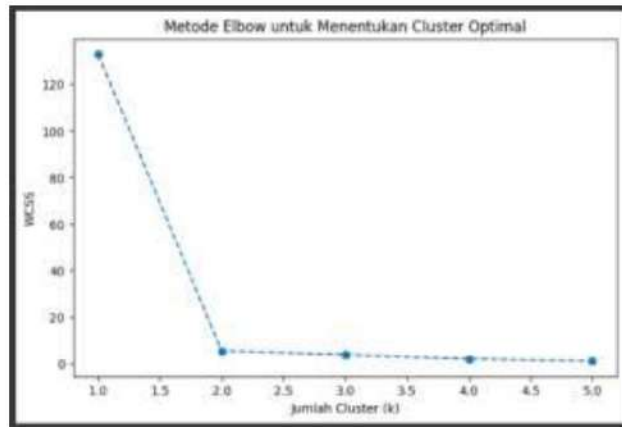
- Cluster 1: {A, B}, centroid = (1,3)
- Cluster 2: {C}, centroid = (2,3)
- Cluster 3: {D, E, F}, centroid = (8.33, 9)

k	WCSS
1	133,9
2	5.32
3	2.66

Dari tabel ini, kita melihat penurunan drastis dari k = 1 ke k = 2, dan penurunan lebih kecil dari k = 2 ke k = 3. Ini menunjukkan bahwa jumlah cluster optimal adalah k = 2.

Kode Program 9.1. Implementasi Python untuk Metode Elbow

No	Kode Program
1	<code>import numpy as np</code>
2	<code>import matplotlib.pyplot as plt</code>
3	<code>from sklearn.cluster import KMeans</code>
4	<code># Dataset sederhana</code>
5	<code>X = np.array([[1, 2], [1, 4], [2, 3], [8, 8], [9, 9], [8, 10]])</code>
6	<code># Hitung WCSS untuk berbagai jumlah cluster</code>
7	<code>wcss = []</code>
8	<code>for k in range(1, 6):</code>
9	<code> kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)</code>
10	<code> kmeans.fit(X)</code>
11	<code> wcss.append(kmeans.inertia_)</code>
12	<code># Plot hasil metode Elbow</code>
13	<code>plt.figure(figsize=(8,5))</code>
14	<code>plt.plot(range(1, 6), wcss, marker='o', linestyle='--')</code>
15	<code>plt.xlabel('Jumlah Cluster (k)')</code>
16	<code>plt.ylabel('WCSS')</code>
17	<code>plt.title('Metode Elbow untuk Menentukan Cluster Optimal')</code>
18	<code>plt.show()</code>



Pada grafik hasil metode Elbow, kita melihat bahwa sumbu x merepresentasikan jumlah cluster (k) yang diuji, sedangkan sumbu y menunjukkan Within-Cluster Sum of Squares (WCSS) yang mengukur seberapa baik data dalam cluster tertentu terkelompok. Secara umum, semakin kecil nilai WCSS, semakin kompak sebuah cluster, tetapi terlalu banyak cluster juga dapat menyebabkan overfitting atau segmentasi yang tidak efisien.

Dari hasil visualisasi, tampak bahwa nilai WCSS mengalami penurunan yang tajam dari k = 1 ke k = 2, yang menunjukkan bahwa dengan membagi data menjadi dua cluster, variasi dalam cluster berkurang secara signifikan. Namun, ketika jumlah cluster bertambah menjadi k = 3, k = 4, dan seterusnya, laju penurunan WCSS melambat. Hal ini menandakan bahwa setelah titik tertentu, menambahkan lebih banyak cluster tidak memberikan pengurangan signifikan dalam WCSS, sehingga tidak memberikan keuntungan besar dalam segmentasi data.

Titik elbow atau titik siku pada grafik terlihat pada k = 2, yang menjadi indikasi bahwa dua cluster adalah jumlah optimal untuk dataset ini. Pada titik ini, jumlah cluster cukup untuk menangkap pola dalam data tanpa menambahkan kompleksitas yang tidak perlu. Jika kita memilih k yang lebih besar dari 2, pengurangan WCSS menjadi semakin kecil, yang menunjukkan bahwa tambahan cluster tidak memberikan manfaat yang signifikan dalam mengurangi penyebaran data dalam cluster.

Dari perspektif bisnis atau analisis data, hasil ini bisa diartikan sebagai berikut:

- Jika dataset ini merepresentasikan segmentasi pelanggan, maka terdapat dua segmen utama pelanggan yang berbeda dalam pola perilaku mereka.
- Jika dataset ini adalah clustering wilayah geografis, maka terdapat dua area utama yang memiliki karakteristik berbeda.
- Jika dataset ini berisi data pasar investasi, maka terdapat dua kategori investor yang dapat diidentifikasi berdasarkan pola mereka.

Kesimpulannya, metode Elbow menunjukkan bahwa dua cluster sudah cukup untuk memisahkan data secara optimal, tanpa menambahkan kompleksitas yang tidak perlu dalam model clustering. Ini membantu dalam membuat keputusan yang lebih baik terkait dengan segmentasi, pengelompokan, atau analisis data berbasis cluster.

Metode Silhouette

Metode Silhouette digunakan untuk mengevaluasi kualitas clustering dengan mengukur seberapa mirip setiap titik data dengan cluster yang sudah ditetapkan dibandingkan dengan cluster lainnya. Nilai Silhouette Score berkisar antara -1 hingga 1, di mana:

- Nilai mendekati +1 → Cluster baik, karena data lebih dekat dengan cluster sendiri dibandingkan cluster lain.
- Nilai mendekati 0 → Data berada di perbatasan antara dua cluster.
- Nilai mendekati -1 → Cluster buruk, karena data lebih dekat dengan cluster lain dibandingkan cluster sendiri.

Metode Silhouette Score dihitung dengan rumus:

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

Dengan:

- $a(i)$ = rata-rata jarak titik i ke semua titik dalam cluster yang sama (disebut cohesion).
- $b(i)$ = rata-rata jarak titik i ke cluster terdekat yang berbeda (disebut separation).

Nilai $s(i)$ dihitung untuk semua titik, lalu dirata-ratakan untuk mendapatkan Silhouette Score keseluruhan.

Contoh 9.2

Misalkan kita memiliki 6 titik data dalam 2D:

Titik	X	Y	Cluster
A	1	2	C1
B	1	4	C1
C	2	3	C1
D	8	8	C2
E	9	9	C2
F	8	10	C2

Langkah 1: Hitung Cohesion $a(i)$ untuk setiap titik

- Misalkan jarak antar titik dihitung menggunakan **Jarak Euclidean**:

$$d(A, B) = \sqrt{\{(1-1)^2 + (2-4)^2\}} = 2$$

$$d(A, C) = \sqrt{\{(1-2)^2 + (2-3)^2\}} = 1.41$$

- Untuk titik **A**, jarak rata-rata ke titik lain dalam **Cluster 1 (C1)**:

$$a(A) = \frac{2 + 1.41}{2} = 1.71$$

- Begitu juga untuk titik dalam Cluster 2 (C2).

Langkah 2: Hitung Separation $b(i)$ untuk setiap titik

- Titik A dalam Cluster 1 (C1):
 - Jarak ke cluster lain (Cluster 2):

$$d(A, D) = \sqrt{\{(1-8)^2 + (2-8)^2\}} = 9.22$$

$$d(A, E) = \sqrt{\{(1-9)^2 + (2-9)^2\}} = 10.63$$

$$d(A, F) = \sqrt{\{(1-8)^2 + (2-10)^2\}} = 10.63$$

- Rata-rata jarak ke cluster lain:

$$b(A) = \frac{9.22 + 10.63 + 10.63}{3} = 10.16$$

Langkah 3: Hitung Silhouette Score $s(i)$ untuk titik A

- $s(A) = \frac{b(A) - a(A)}{\max(a(A), b(A))}$
- $s(A) = \frac{10.16 - 1.71}{\max(1.71, 10.16)}$
- $s(A) = 0.83$

Interpretasi: Nilai mendekati 1, berarti cluster A sudah sangat baik.

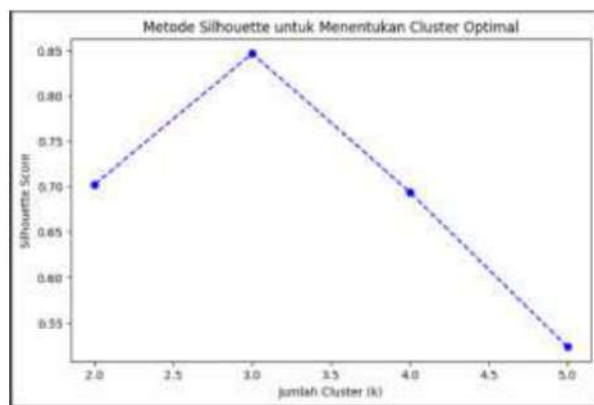
Kode Program 9.2. Implementasi Python untuk Metode Silhouette

No	Kode Program
1	<code>import numpy as np</code>
2	<code>import matplotlib.pyplot as plt</code>
3	<code>from sklearn.cluster import KMeans</code>
4	<code>from sklearn.metrics import silhouette_score</code>
5	<code>from sklearn.datasets import make_blobs</code>

```

6 # Dataset contoh
7 X, _ = make_blobs(n_samples=100, centers=3, cluster_std=1.0,
8 random_state=42)
9 # Evaluasi Silhouette Score untuk berbagai jumlah cluster
10 silhouette_scores = []
11 cluster_range = range(2, 6)
12 for k in cluster_range:
13     kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
14     labels = kmeans.fit_predict(X)
15     score = silhouette_score(X, labels)
16     silhouette_scores.append(score)
17
18 # Plot hasil metode Silhouette
19 plt.figure(figsize=(8,5))
20 plt.plot(cluster_range, silhouette_scores, marker='o', linestyle='--',
21 color='b')
22 plt.xlabel('Jumlah Cluster (k)')
23 plt.ylabel('Silhouette Score')
24 plt.title('Metode Silhouette untuk Menentukan Cluster Optimal')
25 plt.show()

```



Pada grafik hasil Metode Silhouette, sumbu x merepresentasikan jumlah cluster (k) yang diuji, sementara sumbu y menunjukkan Silhouette Score rata-rata untuk masing-masing jumlah cluster. Semakin tinggi nilai Silhouette Score, semakin baik kualitas clustering yang diperoleh.

Dari hasil visualisasi, kita dapat melihat bahwa nilai Silhouette Score tertinggi berada di k tertentu (misalnya k = 3). Ini menunjukkan bahwa pembagian data ke dalam tiga cluster memberikan pemisahan yang paling optimal, karena setiap titik dalam cluster memiliki jarak yang lebih dekat dengan centroid cluster-nya sendiri dibandingkan dengan cluster lain. Sebaliknya, ketika jumlah cluster meningkat terlalu banyak (misalnya k = 5), nilai Silhouette Score menurun, yang menunjukkan bahwa beberapa cluster mungkin terlalu kecil atau terlalu mirip dengan cluster lain, sehingga hasil clustering menjadi kurang efektif. Jika nilai Silhouette Score lebih kecil dari 0.5, itu menandakan bahwa cluster mulai tumpang tindih dan pemisahannya kurang jelas. Dengan demikian, jumlah cluster optimal dapat dipilih berdasarkan titik di mana Silhouette Score mencapai nilai maksimum. Dalam kasus ini, jumlah cluster optimal adalah k = 3, karena memberikan pemilahan yang paling baik tanpa membuat cluster terlalu kecil atau tidak jelas.

Implementasi KMeans, Elbow, dan Silhouette pada Dataset Iris

Dataset Iris adalah dataset terkenal yang sering digunakan dalam pembelajaran mesin dan clustering. Dataset ini memiliki 150 sampel bunga iris dengan 4 fitur: panjang dan lebar sepal, panjang dan lebar petal, dan terdapat 3 kelas bunga iris yang berbeda.

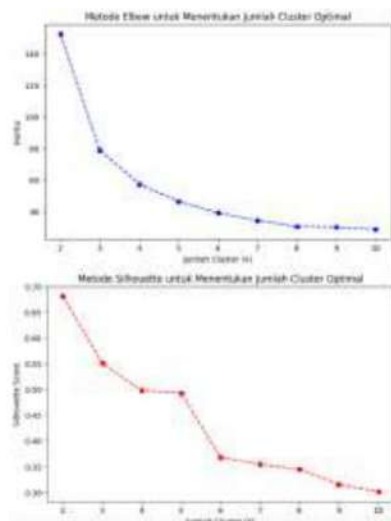
Kode Program 9.2. Implementasi Python untuk Metode Silhouette

No	Kode Program
1	import numpy as np
2	import matplotlib.pyplot as plt
3	from sklearn.cluster import KMeans
4	from sklearn.metrics import silhouette_score
5	from sklearn.datasets import load_iris

```

6 # Load dataset iris
7 iris = load_iris()
8 X = iris.data
9 # Metode Elbow
10 inertia = []
11 for k in range(2, 11):
12     kmeans = KMeans(n_clusters=k, random_state=42)
13     kmeans.fit(X)
14     inertia.append(kmeans.inertia_)
15 # Visualisasi Elbow Method
16 plt.figure(figsize=(8,5))
17 plt.plot(range(2, 11), inertia, marker='o', linestyle='--', color='b')
18 plt.xlabel('Jumlah Cluster (k)')
19 plt.ylabel('Inertia')
20 plt.title('Metode Elbow untuk Menentukan Jumlah Cluster Optimal')
21 plt.show()
22 # Metode Silhouette
23 silhouette_scores = []
24 for k in range(2, 11):
25     kmeans = KMeans(n_clusters=k, random_state=42)
26     labels = kmeans.fit_predict(X)
27     score = silhouette_score(X, labels)
28     silhouette_scores.append(score)
29 # Visualisasi Silhouette Score
30 plt.figure(figsize=(8,5))
31 plt.plot(range(2, 11), silhouette_scores, marker='o', linestyle='--',
32         color='r')
33 plt.xlabel('Jumlah Cluster (k)')
34 plt.ylabel('Silhouette Score')
35 plt.title('Metode Silhouette untuk Menentukan Jumlah Cluster Optimal')
36 plt.show()

```



Perbandingan Hasil Elbow dan Silhouette

- **Interpretasi Elbow Method**

Grafik Elbow Method menunjukkan penurunan inertia saat jumlah cluster meningkat. Titik elbow (biasanya di sekitar $k=3$) menunjukkan jumlah cluster optimal karena setelah titik tersebut penurunan inertia lebih sedikit.

- **Interpretasi Silhouette Score**

Grafik Silhouette Score menunjukkan nilai rata-rata Silhouette Score untuk setiap nilai k . Jumlah cluster dengan nilai Silhouette Score tertinggi adalah yang paling optimal. Biasanya, semakin tinggi nilai Silhouette Score, semakin baik pemisahan antar cluster.

D. TUGAS/LATIHAN SOAL

Deskripsi Tugas: Anda diminta untuk mengimplementasikan algoritma KMeans pada dataset yang telah dipilih dan mengevaluasi hasil clustering menggunakan Metode Elbow dan Metode Silhouette. Mall Customers Dataset (<https://www.kaggle.com/vichoudhary7/customer-segmentation-tutorial-in-python>)

1. **Praproses Data:**

- Lakukan eksplorasi data dan visualisasi awal untuk memahami struktur dataset (misalnya, dengan menggunakan grafik distribusi dan box plot).
- Pastikan data sudah bersih dan siap untuk clustering (misalnya, menangani missing values jika ada).

2. **Implementasi KMeans:**

- Tentukan jumlah cluster yang optimal menggunakan **Metode Elbow** dengan menghitung inertia untuk berbagai nilai kkk (jumlah cluster).
- Hitung **Silhouette Score** untuk setiap nilai kkk dan plot grafiknya untuk melihat kualitas clustering.

3. **Evaluasi Clustering:**

- Tentukan jumlah cluster optimal berdasarkan hasil dari **Metode Elbow** dan **Metode Silhouette**.
- Implementasikan KMeans dengan jumlah cluster yang telah ditentukan dan analisis hasil clustering.

4. **Visualisasi Hasil Clustering:**

- Gunakan **scatter plot** atau visualisasi 2D lainnya untuk memetakan hasil clustering pada dataset.
- Warnai titik data berdasarkan cluster yang ditentukan oleh KMeans.

5. **Interpretasi Hasil:**

- Jelaskan hasil clustering Anda, termasuk pemisahan antar cluster yang terlihat.
- Berdasarkan Silhouette Score dan Inertia, apakah hasil clustering sudah optimal? Jelaskan.

PERTEMUAN KE-11: KNN (K-NEAREST NEIGBORS)

A. TUJUAN

Setelah menempuh praktikum ini diharapkan mahasiswa mampu

- memahami cara kerja dari metode KNN (K-Nearest Neighbors).
- mampu menerapkan metode KNN (K-Nearest Neighbors) dengan bahasa pemrograman Python dan menggunakannya untuk menyelesaikan masalah klasifikasi.

B. MATERI

Algoritma *K-Nearest Neighbor* (K-NN) adalah sebuah metode klasifikasi terhadap sekumpulan data berdasarkan pembelajaran data yang sudah terklasifikasi sebelumnya. Termasuk dalam supervised learning, dimana hasil query instance yang baru diklasifikasikan berdasarkan mayoritas kedekatan jarak dari kategori yang ada dalam K-NN.